

Head Seeker

Overview

As mentioned in T3, protocol processing is a common application for Vitis HLS. In this lab, you are required to design a system that can find the header among the input data stream and export the aligned data stream. Suppose the input data stream is `3siEE21620231121HW5sdEEfsasd3weo` and the header `EE216` occurs at the 3th byte, insert a `' '` before `EE216` and then `EE216` will occur at the 4th byte of the output. The output data stream will look like `3si EE21620231121HW5sdEEfsasd3weo`.

Declaration of the top-level

```
void headseeker(hls::stream<ap_axiu<32,0,0,0> >& msg_in,
hls::stream<ap_axiu<32,0,0,0> >& msg_out)
{
    ap_axiu<32,0,0,0> curr_word;
    ap_axiu<32,0,0,0> send_word;
    ap_uint<32> msg;
    if(!msg_in.empty())
    {
        msg_in.read(curr_word);
    }

    msg = curr_word.data;
    ...
    send_word.data = msg;
    msg_out.write(send_word);
}
```

The interface of the system is `hls::stream` and the data type is `ap_axiu<32,0,0,0>`. The `ap_axiu` can be regarded as a combination of data information and several other common information such as `tlast`, `tuser`, `tkeep` and etc. Although it looks complicated, you don't have to know much about `hls::stream` and `ap_axiu`, just use the given [testbench](#) and figure out how the code shown above works. As shown in the declaration, the system is designed to process 4 bytes (1 word) per cycle. Along with each word (not each byte) is an additional `tlast` signal which indicates whether it is the end of the data stream.

cycle	data	tlast
0	3soi	0
1	EE21	0
2	6202	0
3	3112	0
4	1HW5	0
5	sdEE	0
6	fsas	0
7	d3we	1
8	####	0
9	3soi	0
10	weEE	0
11	2162	0

12		0231		0
13		121H		0
14		w5sd		0
15		EEfs		0
16		asd3		1
17		####		0
18		EE21		0
19		6202		0
20		3112		0
21		1HW5		0
22		sdEE		0
23		fsas		0
24		d3we		0
25		3soi		1
26		####		0
27		soiw		0
28		eEE2		0
29		1620		0
30		2311		0
31		21HW		0
32		5sdE		0
33		Efsa		0
34		sd33		1
35		####		0

The data given above consists of 4 separate data streams, and each of them has eight words and one `EE216`. All the `tlast` signals are 0 except those at the end of each stream, namely `d3we`, `asd3`, `3soi`, `sd33`. Afterward the last word of each data stream is `####`, it can be ignored in the output data stream. And here is an example of the expected output stream:

cycle		data
0		
1		3soi
2		EE21
3		6202
4		3112
5		1HW5
6		sdEE
7		fsas
8		d3we
9		
10		3soi
11		we
12		EE21
13		6202
14		3112
15		1HW5
16		sdEE
17		fsas
18		d3
19		EE21
20		6202
21		3112
22		1HW5
23		sdEE

```
24 | fsas
25 | d3we
26 | 3soi
27 |
28 | soiw
29 | e
30 | EE21
31 | 6202
32 | 3112
33 | 1HW5
34 | sdEE
35 | fsas
36 | d33
```

Requirements

In this lab, you are required to

1. Design an FSM and accomplish the functionality.
2. Guarantee your codes generate the right output data stream and pass the co-simulation.
3. Try to optimize the system throughput and the clock period. In this lab, you should choose the zcu104 board and set the frequency to 400MHz when creating a project. First, try to optimize the initial interval(`II`) of the system as small as possible. If you optimize the initial interval to 1, the estimated clock period may exceeds the target (2.5ns), just ignore it and then try to optimize the estimated clock period.
4. Record the problems you encountered as well as **the effort to optimize** the timing.

Submission: Deadline on 2025-12-7 23:59 PM

1. headseeker.cpp (no headseeker.h & headseeker_tb.cpp, you **MUST** use the given testbench)
2. Report.pdf

Please compress all the files into a ZIP file with name **HW5_YourName_StudentNumber.zip**.
The file structure should be like:

```
HW5_蒋晨昕_2024231020.zip
├─ headseeker.cpp
└─ Report.pdf
```

Overdue Policy

- Submit on time, get all scores.
- Submit within 24 hours of the deadline, get half of the score.
- Submit more than 24 hours after the deadline, get 0 score.